RAJALAKSHMI ENGINEERING
COL LE GE
An AutonomousInstitution

BONAFIDE CERTIFICATE

Name: .....

Academic Year: ..... Semester: ..... Branch: .....

Register No

Certified that this is the bonafide record of work lone by the above student in the ..... Laboratory

during the academic year 202 - 202

Signature of Faculty in-charge

Submitted for the Practical Examination held on.....

Internal Examiner

External Examine

DEPARTMENT OF INFORMATION TECHNOLOGY

LAB MANUAL

CS23432 – Software Construction

(REGULATION 2023)

RAJALAKSHMI ENGINEERING COLLEGE

Thandalam, Chennai-602015

Name:

Register No:

Year / Branch / Section:

Semester:

Academic Year:

INDEX

Exp. NO: 1 AZURE DEVOPS ENVIRONMENT SETUP.

AIM:

To set up and access the Azure DevOps environment by creating an organization through the Azure portal for the Event Management System.

Installation / Procedure:

1. Open your web browser and navigate to the Azure portal:
   https://portal.azure.com
2. Sign in using your Microsoft account credentials.
   - If you do not have an account, create a new Microsoft account.
3. After logging in, the Azure home page will be displayed.
4. In the search bar at the top, type:
   "Azure DevOps Organizations"
5. Click on the Azure DevOps Organizations service from the search results.
6. Click on "My Azure DevOps Organization".
7. Click on "Create Organization"
8. Enter the required details:
   - Organization Name
   - Region.

9. click continue and complete the setup.

10. After successful creation, you will be redirected to the Azure DevOps organization dashboard.

DESCRIPTION:

Azure DevOps provides a cloud-based platform to manage software development activities such as planning, tracking, and collaboration. In this experiment, an organization is created to manage the Event Management System Project.

RESULT:

Successfully accessed the Azure DevOps environment and created a new organization for the Event Management System.

EXP. NO : 2

AZURE DEVOPS PROJECT SETUP AND USER STORY MANAGEMENT

AIM :

To setup an Azure DevOps project for efficient collaboration and agile work management for the Event Management System.

PROCEDURE :

1. Create Azure Account

   Sign in to Azure DevOps using Microsoft credentials.

2. Create New project

   a. Go to organization home page

   b. click on "New project"

   c. Enter details :

      - Name : Event Management System
      - Description : A system to manage events, bookings, and users
      - visibility : Private

   d. click create.

3. Project Dashboard

   * After creation, project dashboard will be displayed.
   * It includes Boards, Repos, Pipelines, etc..

4. Create User Stories

a. Go to Boards → Work Items

b. Click New Work Item → User story

c. Enter details :

Sample User stories :

* As a user, I want to register, so that I can use the system
* As a user, I want to login, so that I can access event features.
* As a user, I want to view events, so that I can choose events

DESCRIPTION :

User stories represent system requirements from the user's perspective and help in agile development.

RESULT :

Successfully created Azure DevOps project and added user stories for Event Management System.

Exp. NO : 3

SETTING UP EPICS, FEATURES AND USER STORIES

AIM :

TO create Epics, Features, User Stories, and Tasks for the Event Management System.

Create Epics :

1. Go to Boards → Backlogs
2. Select Epics level
3. Click New Epic

Epics Created :

* User Management
* Event Management
* Booking system
* Payment system
* Admin Dashboard

Create Features :

1. Switch to Feature level
2. Add features under each Epic

Features :

User Management

- Registration
- Login

Event Management

- Create Event
- Edit Event
- Delete Event

Booking

- Book Ticket
- Cancel Ticket

Payment

- Online Payment Integration

Admin

- View Reports
- Manage Users

Create User Stories:

* As a user, I want to login so that I can access the system.
* As a user, I want to book tickets so that I can attend events.
* As an admin, I want to manage events so that system runs smoothly.

Create Tasks:

* Design UI pages
* Develop backend APIs
* Database integration
* Testing and debugging

DESCRIPTION:

Epics divide the project into major sections, features define functionalities, and user stories describe specific user requirements.

RESULT:

Epics, Features, User stories, and Tasks are successfully created.

Exp. NO : 4 SPRINT PLANNING

AIM :

To align user stories to specific sprints for the Event Management System.

SPRINT PLANNING :

Sprint 1 (User Module)

- User Registration
- User Login

Sprint 2 (Event Module)

- Create Event
- View Events

Sprint 3 (Booking Module)

- Book Ticket
- Cancel Ticket.

Sprint 4 (Payment + Admin)

- Payment Integration
- Admin Dashboard

STEPS :

1. Go to Boards → Sprints
2. Create new sprint
3. Assign user stories to sprint
4. Set iteration dates.

DESCRIPTION :

Sprint planning is used in Agile methodology to divide work into manageable time frames for efficient development.

RESULT :

Sprints are successfully created and user stories are assigned. *

Exp.NO : 5 POKER ESTINATION

AIM:

To estimate user stories using planning poker technique.

PROCEDURE:

1. Identify all user stories
2. Use Fibonacci Series values (1, 2, 3, 5, 8)
3. Assign story points based on complexity
4. Discuss with team members
5. Finalize estimation.

Story Points Table:

user story	story points
User Registration	3
Login	2
Create Event	5
Book Ticket	5
Payment integration	8
Admin Dashboard	8

EXPLANATION :

Planning poker is an agile estimation technique where team members assign story points based on effort, complexity, and risk.

DESCRIPTION :

Poker Estimation is an agile technology used to estimate workload collaboratively and improve planning accuracy.

RESULT :

User stories are successfully estimated using Planning Poker technique.

Exp. No: 6

DESIGNING CLASS AND SEQUENCE DIAGRAM FOR PROJECT ARCHITECTURE

AIM:

To Design a class Diagram and sequence diagram for the Event Management system to represent system structure and interactions

1. CLASS DIAGRAM:

A class diagram represents the static structure of the system by showing classes, attributes, methods and relationships.

1. user class:

Attributes:
* userId
* name
* email
* password

methods:
* register()
* login()
* updateProfile()

2. Event class:

Attributes:
* eventId
* eventName
* date
* venue
* price

Methods:
* createEvent()
* updateEvent()
* deleteEvent()

3. Booking class :

Attributes :

* bookingId
* userId
* eventId
* bookingDate

Methods :

* bookTicket()
* cancelBooking()

4. Payment class :

Attributes :

* paymentId
* amount
* paymentStatus

Methods :

* processPayment()
* verifyPayment()

5. Admin class :

Attributes :

* adminId

Methods :

* manageUsers()
* manageEvents()
* viewReports()

Relationships :

* User → Booking (One-to-Many)
* Event → Booking (One-to-Many)
* Booking → Payment (One-to-One)
* Admin → Event (Manages)

2. SEQUENCE DIAGRAM

Definition:

A sequence Diagram represents the interaction between objects in a system in a time sequence.

Actors involved:

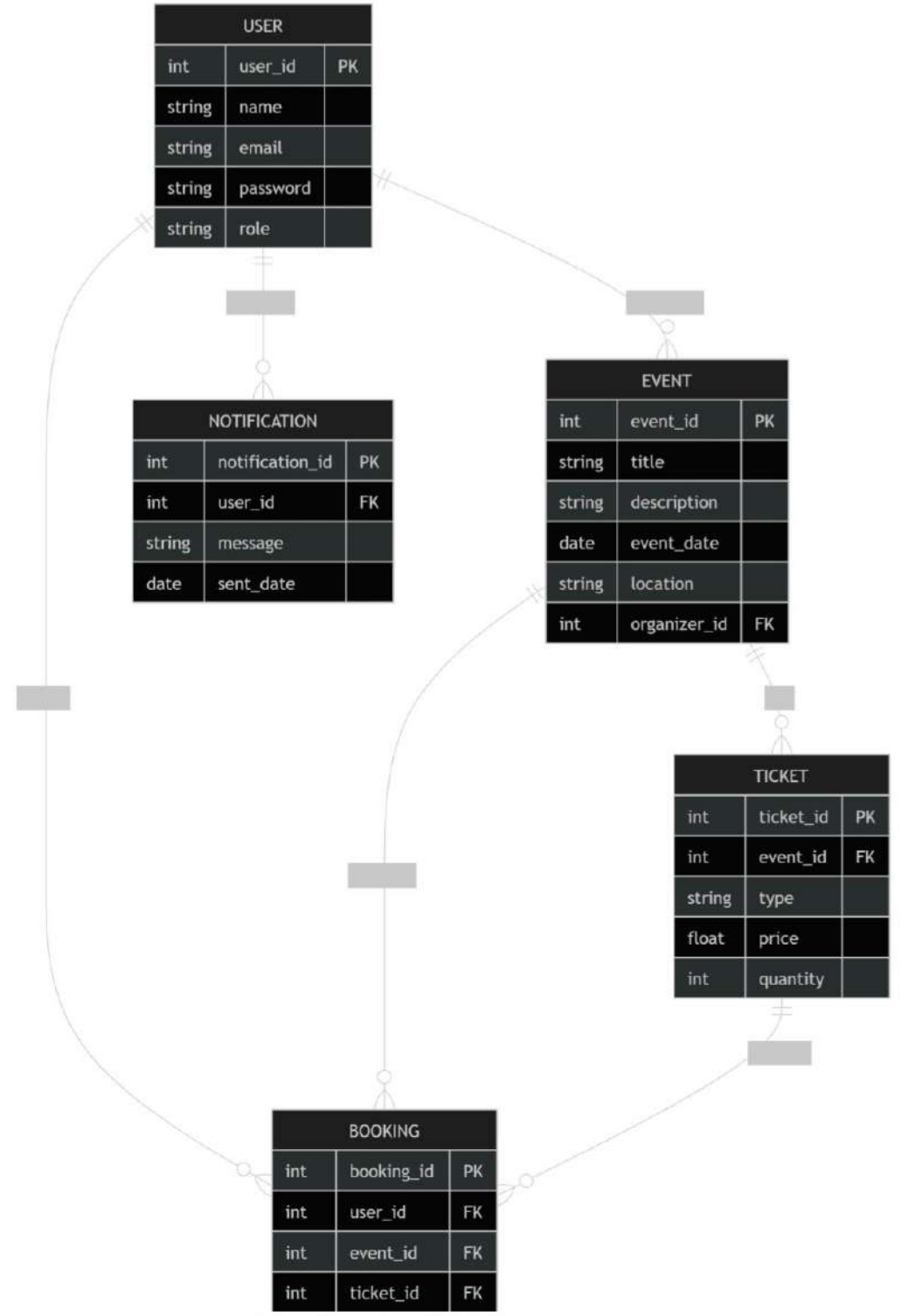
* User
* System
* Payment Gateway

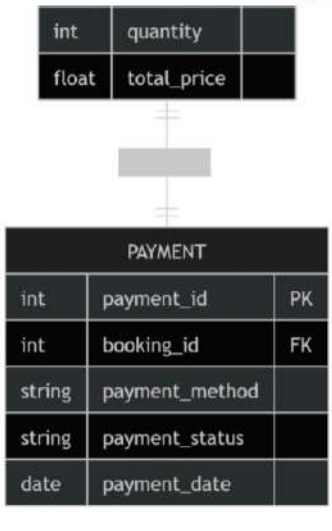
Sequence Flow:

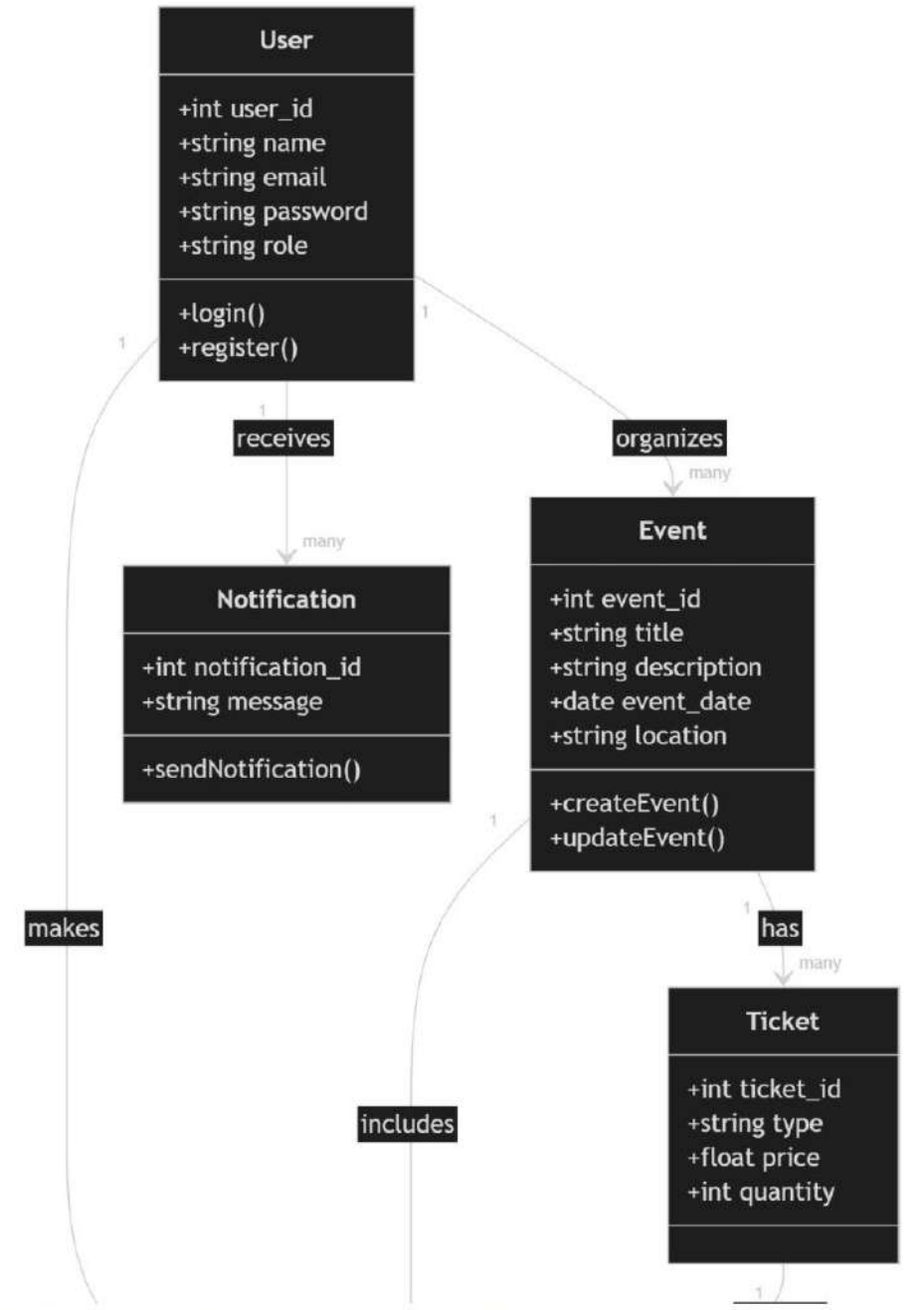
1. User logs into the system.
2. System validates login credentials
3. User browses available events
4. System displays event list
5. User selects an event
6. User clicks on "Book Ticket"
7. Booking request is sent to the system
8. System process booking request
9. Payment request is sent to payment gateway
10. Payment gateway process payment
11. Payment success response is sent
12. System confirms booking
13. confirmation message displayed to user

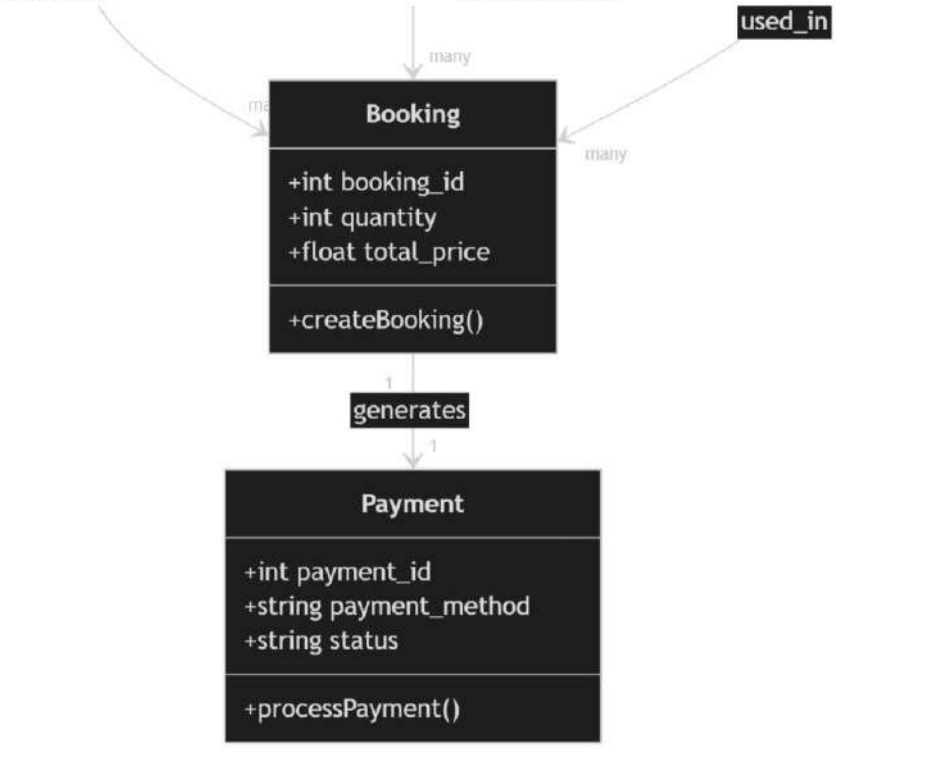
RESULT:

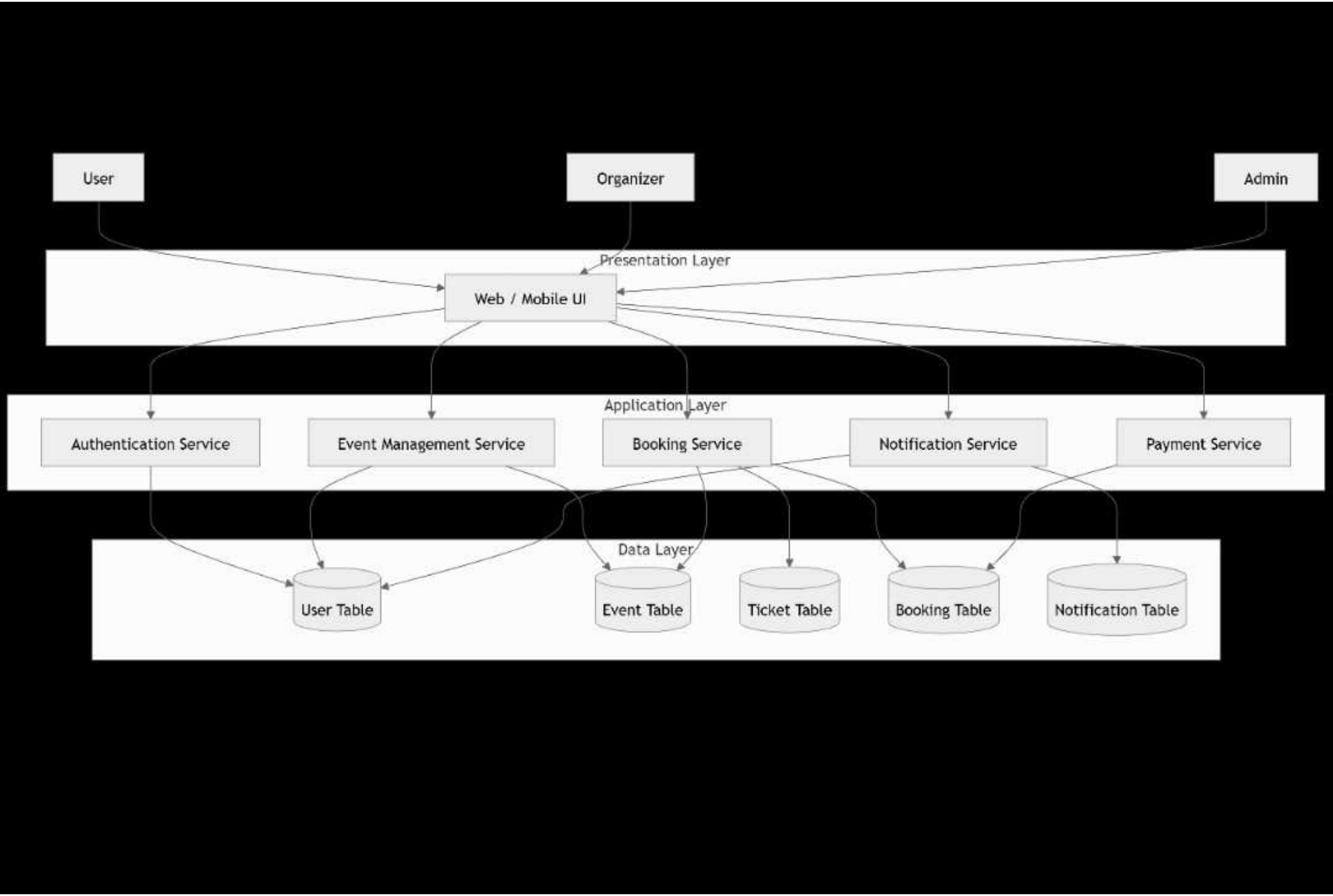
The class and sequence diagram is designed successfully for the Event management system.


USER
int user_id PK
string name
string email
string password
string role
NOTIFICATION
int notification_id PK
int user_id FK
string message
date sent_date
EVENT
int event_id PK
string title
string description
date event_date
string location
int organizer_id FK
TICKET
int ticket_id PK
int event_id FK
string type
float price
int quantity
BOOKING
int booking_id PK
int user_id FK
int event_id FK
int ticket_id FK


int quantity
float total_price
PAYMENT
int payment_id PK
int booking_id FK
string payment_method
string payment_status
date payment_date

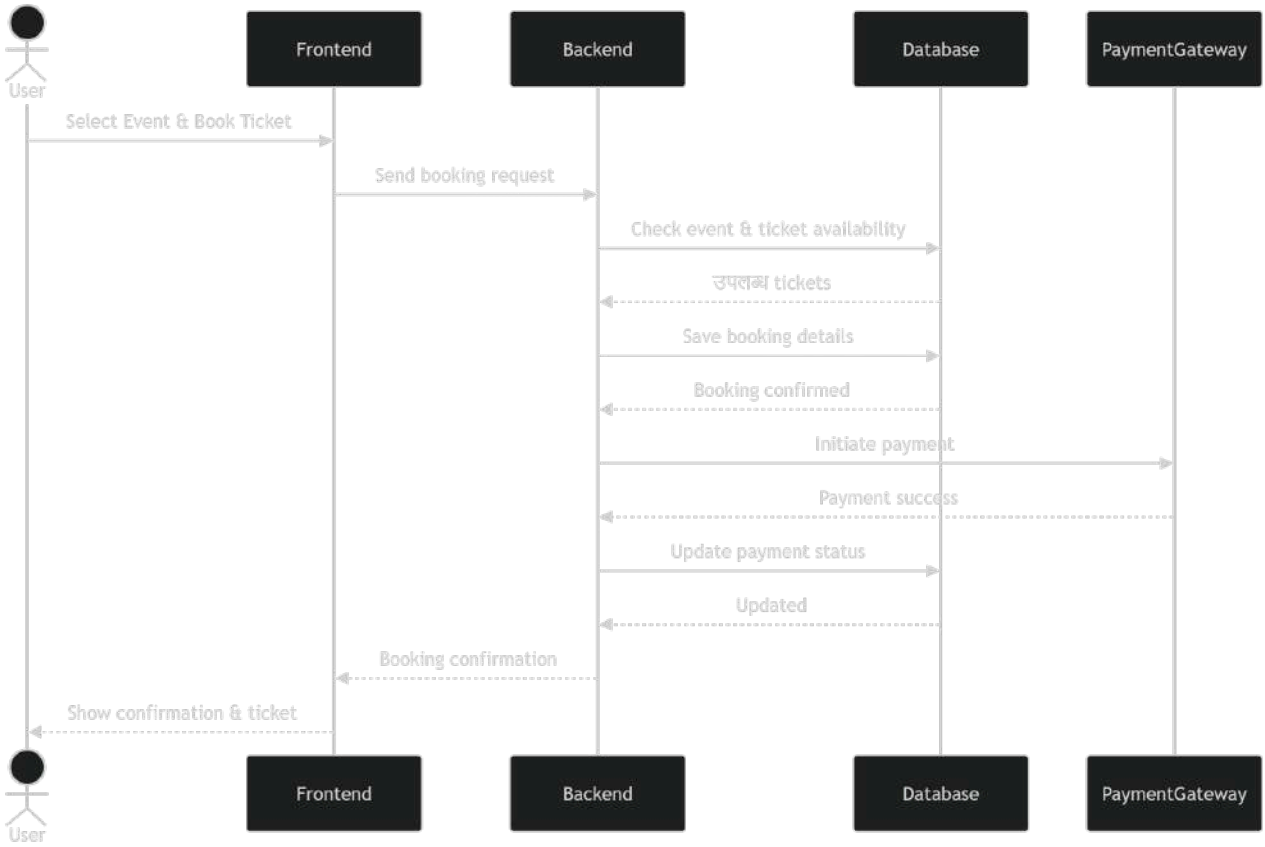

User
+int user_id
+string name
+string email
+string password
+string role
+login()
+register()
receives
organizes
Notification
+int notification_id
+string message
+sendNotification()
Event
+int event_id
+string title
+string description
+date event_date
+string location
+createEvent()
+updateEvent()
makes
has
Ticket
+int ticket_id
+string type
+float price
+int quantity
includes
1
many


used_in
Booking
+int booking_id
+int quantity
+float total_price
+createBooking()
generates
Payment
+int payment_id
+string payment_method
+string status
+processPayment()
many
1


User
Organizer
Admin
Presentation Layer
Web / Mobile UI
Application Layer
Authentication Service
Event Management Service
Booking Service
Notification Service
Payment Service
Data Layer
User Table
Event Table
Ticket Table
Booking Table
Notification Table

Sequence Diagram

Last updated by | suriya kr | Apr 8, 2026 at 11:17 PM GMT+5:30



): 7 DESIGNING ARCHITECTURAL AND ER DIAGRAMS FOR PROJECT STRUCTURE

AIM :

TO Design an Architectural Diagram & ER diagram for the event management system.

ARCHITECTURAL DIAGRAM :

components :

1. User / Organizer / Admin
   * Different roles interacting with the system
   * Users : Book events
   * Organizers : create / manage events
   * Admin : Overall system control
2. Frontend (web App / Mobile App)
   * User interface layer
   * Technologies : HTML, CSS, Javascript / Mobile Apps
   * Handles :
     - Event browsing
     - Registration & Login
     - Ticket booking
3. Azure API management
   * Acts as a gateway for all API requests
   * secures manages communication between frontend & backend.

4. Azure App service:

* core backend service
* Handles:
  - Business logic
  - Event processing
  - Booking system

5. Azure Active Directory:

* Authentication and authorization
* Ensures secure login for users, organizers, and admins.

6. Azure Functions:

* Serverless computing
* Handles background tasks like:
  - Notification
  - Email confirmations
  - Event reminders

7. Azure SQL database:

* Stores structured data:
  - Users
  - Events
  - Booking
  - Payments.

8. Azure Blob storage:

* Stores unstructured data:
  - Event images
  - Documents
  - Media files.

9. Payment Gateway:

* Handles online transactions
* Secure payment processing

10. Azure Monitor:

* Tracks system performance
* Logs errors and usage analytics

Architecture Flow:

User → Frontend → API management → App service → (Database / storage / Functions / Authentication / payment) → Azure monitor (for tracking)

2. ER DIAGRAM:

Entities:

1. User
   * User_ID (PK)
   * Name
   * Email
   * Password
   * Role
2. Event
   * Event_ID (PK)
   * Event_Name
   * Date
   * Location
   * Organizer_ID

3. Booking:

* Booking_ID (PK)
* User_ID (FK)
* Event_ID (FK)
* Booking_Date
* Status

4. Payment:

* Payment_ID (PK)
* Booking_ID (FK)
* Amount
* Payment_Status
* Payment_mode

5. Organizer:

* Organizer_ID (PK)
* Name
* contact

Relationships:

* One User → Many Bookings (1:M)
* One Event → Many Bookings (1:M)
* One Booking → One Payment (1:1)
* One Organizer → Many Events (1:M)

RESULT:

The Architectural Diagram and ER diagram for Event management system were successfully designed and implemented.

Exp.NO:8 TESTING - TEST PLAN AND TEST CASES

AIM:

To design the Test Plan and Test cases for validating the Event Management System.

TEST PLAN:

1. Objective

To ensure that all modules of the Event Management System work correctly and meet user requirements.

2. Scope

Testing includes the following modules:

* User Registration & Login
* Event creation & management
* Event Booking
* Payment Processing

3. Testing type

* Functional testing
* Integration testing
* System testing

4. Test Environment

* OS: Windows / Linux
* Browser: chrome / Edge
* Database: MySQL

5. Test strategy :

* Validate inputs and outputs
* check database updates
* Verify user interface behavior

TEST CASES :

Test case 1 : User Registration

Test case ID	TC-01
Description	Register a new user
Input	Name, Email, Password
Expected output	User registered successfully
Actual output	User registered successfully
Status	Pass

Test case 2 : User Login

Test case ID	TC-02
Description	Login with valid credentials
Input	Valid Email & password
Expected output	Login successful
Actual output	Login successful
Status	Pass

Test case 3 : Invalid login

Test case ID	TC-03
Description	login with invalid credentials
Input	Wrong Email / Password
Expected Output	Error message displayed
Actual Output	Error message displayed
Status	Pass

Test case 4 : Create Event

Test case ID	TC-04
Description	Organizer creates an event
Input	Event details
Expected Output	Event created successfully
Actual Output	Event created successfully
Status	Pass

Test case 5 : Book Event

Test case ID	TC-05
Description	user books an event
Input	Event selection
Expected output	Booking confirmed
Actual output	Booking confirmed
Status	Pass

RESULT :

The test plan and test cases for the Event Management system were successfully designed & validated.

Exp. No : 9

LOAD TESTING AND PIPELINES

AIM:

To perform load testing and create a CI/CD pipeline for the Event Management system using Azure DevOps

Load Testing:

Azure load testing is used to evaluate the performance of the event management system when many users access it simultaneously.

Tested APIs:

* Book Event Ticket
* View Events
* Payment Processing

Steps:

* create Azure Load Testing resource
* Add HTTP test (Event booking API)
* configure virtual users (50-100 users)
* Run test and monitor response time, errors and throughput.

Observation:

* Increased users may slow down booking process
* payment module may take more response time under load.

Pipeline:

Pipeline automates the build, testing and deployment of the Event Management System. It ensures continuous integration and faster delivery of updates.

Steps:

* connect GitHub repository to Azure Devops
* create azure-pipelines.yml file.

Add tasks:

* Install dependencies
* Build and run application
* Execute basic tests.

Advantages:

* Faster and automated deployment
* Early detection of errors
* improved code quality

Result:

Load testing and pipeline setup for the Event Management System were successfully completed, ensuring performance and automation.

Exp.NO:10

GITHUB PROJECT STRUCTURE & NAMING

AIM:

TO organize the Event Management System project using a proper folder structure and naming conventions for better development and maintenance.

Project structure:

The project is divided into frontend and backend, with backend further organized into modular folders.

```
eventmanage/
    frontend/
    server/
        config/
        controllers/
        middleware/
        models/
        routes/
        public/
        uploads/
        node_modules/
        .env
        package.json
        package-lock.json
        server.js
        seeder.js
        update-porters.js
    README.md
```

Naming conventions :-

* Files → lowercase with hyphens or camelcase
  eg : server.js , update-parkers.js
* Folders → lowercase
  eg : controllers , models
* classes → Pascal case
  eg : EventManage
* Variables → camel case
  eg : eventId , userData
* APIs → REST formal
  eg : /api/events
  /api/users

Best Practices:

* Seperate frontend and backend clearly
* Follow MVC architecture (Model , controller , Routes)
* Store sensitive data in .env file
* Keep reusable logic in middleware
* Use meaningful & consistent naming
* Maintain modular & scalable code structure.

RESULT :

The Event Management system project is well-structured and easy to manage , improving development efficiency and scalability.